

DAY1- CASE STUDY 3: Library Book Management System

Case Study Question

A library is implementing a Book Management System using MongoDB.

The system should:

- Create a database to store book information.

```
> use libraryDB
< switched to db libraryDB
libraryDB> |
```

- Create a collection to store book records.

```
> db.createCollection("books")
< { ok: 1 }
```

- Insert one book record when a new book is purchased.

```
> db.books.insertOne({ bookId: 1, title: "MongoDB Basics", author: "Tom", price: 450, category: "Database" })
```

- Insert multiple book records when importing book data from an old catalog system.

```
> db.books.insertMany([
  { bookId: 2, title: "Java Fundamentals", author: "James", price: 500, category: "Programming" },
  { bookId: 3, title: "Python Guide", author: "Guido", price: 480, category: "Programming" },
  { bookId: 4, title: "Data Science", author: "Andrew", price: 700, category: "Analytics" },
  { bookId: 5, title: "Web Development", author: "Mark", price: 550, category: "Web" },
  { bookId: 6, title: "AI Basics", author: "Alan", price: 800, category: "AI" },
  { bookId: 7, title: "Cloud Computing", author: "Jeff", price: 650, category: "Cloud" },
  { bookId: 8, title: "Cyber Security", author: "Kevin", price: 600, category: "Security" },
  { bookId: 9, title: "Big Data", author: "Hadoop", price: 750, category: "Analytics" },
  { bookId: 10, title: "SQL Mastery", author: "Oracle", price: 400, category: "Database" }
])
```

- Ensure that duplicate book IDs are handled correctly during bulk insert operations.

```
> db.books.insertMany([
  { bookId: 2, title: "Java Fundamentals", author: "James", price: 500, category: "Programming" },
  { bookId: 3, title: "Python Guide", author: "Guido", price: 480, category: "Programming" },
  { bookId: 4, title: "Data Science", author: "Andrew", price: 700, category: "Analytics" },
  { bookId: 5, title: "Web Development", author: "Mark", price: 550, category: "Web" },
  { bookId: 6, title: "AI Basics", author: "Alan", price: 800, category: "AI" },
  { bookId: 7, title: "Cloud Computing", author: "Jeff", price: 650, category: "Cloud" },
  { bookId: 8, title: "Cyber Security", author: "Kevin", price: 600, category: "Security" },
  { bookId: 9, title: "Big Data", author: "Hadoop", price: 750, category: "Analytics" },
  { bookId: 10, title: "SQL Mastery", author: "Oracle", price: 400, category: "Database" }
],{ordered:false})
```

- Retrieve all book records for inventory checking.

```
> db.books.find()
```

- Retrieve a specific book using the book ID.

```
> db.books.find({ bookId: 3 })
```

Filter books based on:

Category

```
> db.books.find({ category: "Programming" })
```

Author

```
> db.books.find({ author: "Tom" })
```

Price range

```
> db.books.find({ price: { $gt: 450, $lt: 700 } })
```

- Display only selected fields (such as title and price) for library users.

```
> db.books.find({}, { title: 1, price: 1, _id: 0 })
```

- Drop the collection or database when migrating to a new library system.

```
> db.books.drop()
< true
> db.dropDatabase()
< { ok: 1, dropped: 'libraryDB' }
```

🏠 books						
	_id ObjectId	bookId Int32	title String	author String	price Int32	category String
1	ObjectId('6a1034ce582830...	1	"MongoDB Basics"	"Tom"	450	"Database"
2	ObjectId('6a103523582830...	2	"Java Fundamentals"	"James"	500	"Programming"
3	ObjectId('6a103523582830...	3	"Python Guide"	"Guido"	480	"Programming"
4	ObjectId('6a103523582830...	4	"Data Science"	"Andrew"	700	"Analytics"
5	ObjectId('6a103523582830...	5	"Web Development"	"Mark"	550	"Web"
6	ObjectId('6a103523582830...	6	"AI Basics"	"Alan"	800	"AI"
7	ObjectId('6a103523582830...	7	"Cloud Computing"	"Jeff"	650	"Cloud"
8	ObjectId('6a103523582830...	8	"Cyber Security"	"Kevin"	600	"Security"
9	ObjectId('6a103523582830...	9	"Big Data"	"Hadoop"	750	"Analytics"
10	ObjectId('6a103523582830...	10	"SQL Mastery"	"Oracle"	400	"Database"